LESSON 04 · CHEAT SHEET

Method References

All forms since Java 8 · record accessors Java 16
results use the shared course dataset

Key rules

1. **A method reference is a lambda that calls one existing method.** Same behaviour, shorter form. 8
2. **The target type decides** how many parameters there are and where they go.
3. **Static** `Class::m`: all parameters → arguments.
4. **Bound** `obj::m`: receiver fixed; parameters → arguments. Receiver evaluated **once, at creation**; null → NPE at creation.
5. **Unbound** `Class::m` (instance method): **1st parameter = receiver**, the rest → arguments.
6. **Constructor** `Class::new`: parameters → constructor arguments. **Array** `Type[]::new`: int → length.
7. **Keep a lambda** for extra arguments, chains, ambiguous overloads, or when it is clearer.

Canonical snippets

1 · One of each in a real pipeline

```
payments.stream()
    .filter(Fees::isSuccessful)  // static
    .sorted(Comparator.comparing(
            Payment::amount))      // unbound
    .map(Receipt::from)          // static
    .forEach(printer::print);    // bound
ids = s.toArray(String[]::new);   // array
```

2 · Unbound with two parameters

```
BinaryOperator<BigDecimal> ADD = BigDecimal::add;
// (a, b) -> a.add(b): a = receiver, b = argument
accounts.stream().map(Account::balance)
        .reduce(BigDecimal.ZERO, ADD);  // 300382.25
```

3 · Constructor with 5 parameters

```
@FunctionalInterface interface CustomerFactory {
    Customer create(String id, String name,
        String city, Tier tier, LocalDate joined);
}
CustomerFactory NEW_CUSTOMER = Customer::new;
```

Real compiler errors

- `Function<Integer,String> f = Integer::toString;`
  → reference to toString is ambiguous (static toString(int) and instance toString())
- `Function<String,Customer> f = Customer::new;`
  → invalid constructor reference

Reference or lambda?

✔ Reference when

- the lambda only forwards
- the name tells the intent
- record accessor / constructor
- you extracted a long lambda

✘ Keep lambda when

- extra argument: `a -> fee(a, RATE)`
- a chain + comparison
- ambiguous overload
- the lambda is clearer

Mini diagram: where do the parameters go?

static all → args · **bound** obj fixed · **unbound** 1st = receiver · **::new** → constructor

Mapping table: reference → target type → equivalent lambda (BD = BigDecimal)

Kind	Reference	Target type	Equivalent lambda
Static	`Integer::parseInt`	`Function<String,Integer>`	`s -> Integer.parseInt(s)`
Static	`Fees::fee`	`BiFunction<BD,BD,BD>`	`(a, r) -> Fees.fee(a, r)`
Bound	`printer::print`	`Consumer<Receipt>`	`r -> printer.print(r)`
Bound	`"SGD"::equals`	`Predicate<String>`	`s -> "SGD".equals(s)`
Bound	`System.out::println`	`Consumer<String>`	`s -> System.out.println(s)`
Unbound	`Transaction::amount`	`Function<Transaction,BD>`	`t -> t.amount()`
Unbound	`String::toUpperCase`	`Function<String,String>`	`s -> s.toUpperCase()`
Unbound	`BigDecimal::add`	`BinaryOperator<BD>`	`(a, b) -> a.add(b)`
Constructor	`BigDecimal::new`	`Function<String,BD>`	`s -> new BigDecimal(s)`
Constructor	`ArrayList::new`	`Supplier<List<T>>`	`() -> new ArrayList<>()`
Array	`String[]::new`	`IntFunction<String[]>`	`n -> new String[n]`

Lesson 04 · sticky notes

Cut out, stick on your wall, and read one every morning.

:: = call ONE method

A method reference is a lambda that only forwards to one method.

4 kinds + arrays

static · bound · unbound · constructor

`String[]::new`

Unbound

1st parameter = receiver.

`BigDecimal::add` = (a, b) -> a.add(b)

Bound = eager

`factory()::print` calls factory() ONCE, at creation.

null receiver → NPE now.

Records = free refs

`Transaction::amount`

`Account::balance` [16]

Integer::toString ✕

Ambiguous!

Use `i -> i.toString()` or `String::valueOf`

5 params? Own SAM

`CustomerFactory f = Customer::new;`

Clarity wins

Extra argument or chain? Keep the lambda.

Good method names make refs worth it.